

六、实验六 单周期 CPU 实现

1 实验目的

1. 理解 MIPS 指令结构，理解 MIPS 指令集中常用指令的功能和编码，学会对这些指令进行归纳分类。
2. 了解熟悉 MIPS 体系的处理器结构，如延迟槽，哈佛结构的概念。
3. 熟悉并掌握单周期 CPU 的原理和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力。
5. 为后续设计多周期 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习 MIPS 指令集，深入理解常用指令的功能和编码，并进行归纳确定处理器各部件的控制码，比如使用何种 ALU 运算，是否写寄存器堆等。
2. 确定自己本次实验中的准备实现的 MIPS 指令，要求至少实现一条 load 指令、一条 store 指令、10 条基础运算指令、一条跳转指令。其中基础运算指令最好包含多种类型的操作，必须包含一条加法和一条减法指令。不考虑指令可能产生异常的情况。单周期 CPU 的实验重点是搭建出一个 CPU 架构，为避免被繁琐的指令所困惑，建议在单周期 CPU 实验中只实现十几条指令。
3. 对准备实现的指令进行分析，完成表 7.1 的填写。

表 7.1 mips 基础指令特性归纳表

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	目的寄存器	功能描述
R 型指令	addu rd, rs, rt	000000 _{rs} _{rt} rd 000001	[rs]	[rt]	rd	GPR[rd]= GPR [rs]+ GPR [rt]
I 型指令	addiu rt,rs,imm	001001 _{rs} _{rt} imm	[rs]	sign_ext(imm)	rt	GPR [rt]= GPR [rs]+sign_ext(imm)
J 型指令	j target	000010 target	PC	target		跳转, PC={PC[31:28],target,2'b00}

注：GPR 表示通用寄存器，[rs]表示寄存器 rs 里存储的值，PC 表示程序计数器；imm 为 16 位立即数，sign_ext(imm)表示对其进行符号扩展；target 为 26 位立即数。

4. 自行设计本次实验的方案，画出结构框图，大致结构框图如图 7.1。图 7.1 中粗线表示接口位数和种类不定，需要在自己的结构框图中详细给出。从图 7.1 中可以看出，本次实验是需要用到之前实验的结果的，比如 ALU 模块、寄存器堆模块、指令 ROM 模块和数据 RAM 模块，其中 ROM 和 RAM 要使用自行搭建的异步存储器。

单周期 CPU 是指一条指令的所有操作在一个时钟周期内执行完。设计中所有寄存器和存储器都是异步读同步写的，即读出数据不需要时钟控制，但写入数据需时钟控制。

故单周期 CPU 的运作即：在一个时钟周期内，根据 PC 值从指令 ROM 中读出相应的指令，将指令译码后从寄存器堆中读出需要的操作数，送往 ALU 模块，ALU 模块运算得到结果。

如果是 store 指令，则 ALU 运算结果为数据存储的地址，就向数据 RAM 发出写请求，在下一个时钟上升沿真正写入到数据存储器。

如果是 load 指令，则 ALU 运算结果为数据存储的地址，根据该值从数据存 RAM 中读出数据，送往寄存器堆根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中。

如果非 load/store 操作，若有写寄存器堆的操作，则直接将 ALU 运算结果送往寄存器堆根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中。

如果是分支跳转指令，则是需要将结果写入到 pc 寄存器中的。

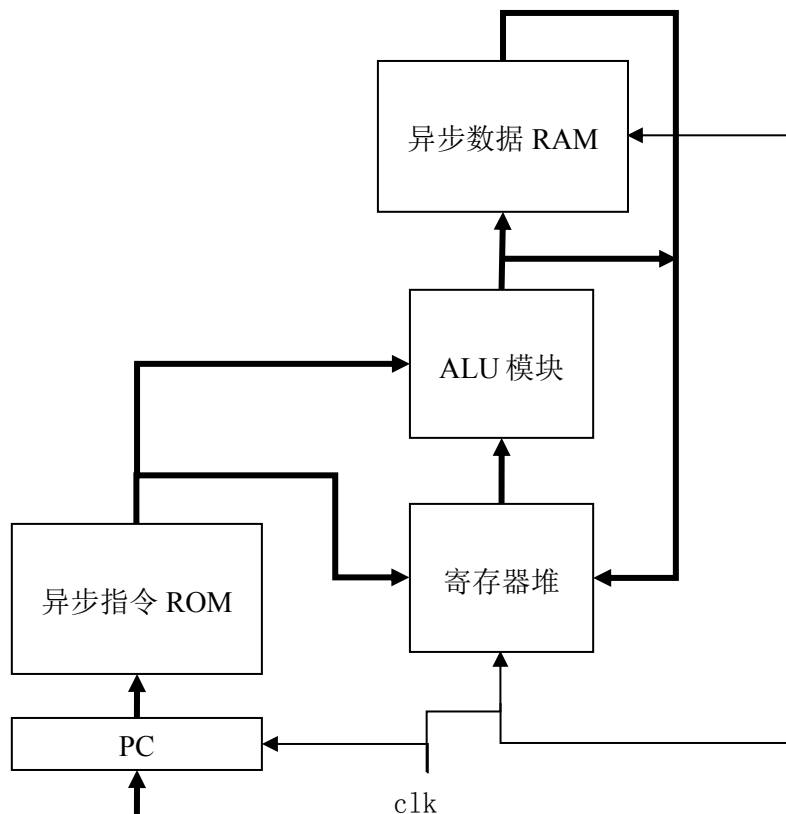


图 7.1 单周期 CPU 的大致框图

5. 根据设计的实验方案，使用 verilog 编写相应代码。
6. 依据自己设计中实现的指令，编写一段不少于 20 行的汇编程序，力求验证到所有实现的指令。该段汇编程序是需要内嵌到自行搭建的异步指令 ROM 中的。完成表 7.2 的填写。

表 7.2 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$t1, \$0, #1	[\$t1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001

7. 对编写的代码进行仿真，得到正确的波形图。

8. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 7.2。外围模块中需调用封装好的 LCD 触摸屏模块，观察单周期 CPU 的内部状态，比如 32 个寄存器的值，PC 的值等。并且需要利用触摸功能输入特定数据 RAM 地址，从该 RAM 的调试端口读出数据显示在屏上，以达到实时观察数据存储器内部数据变化的效果。通过这些手段，可以在板上充分验证 CPU 的正确性。

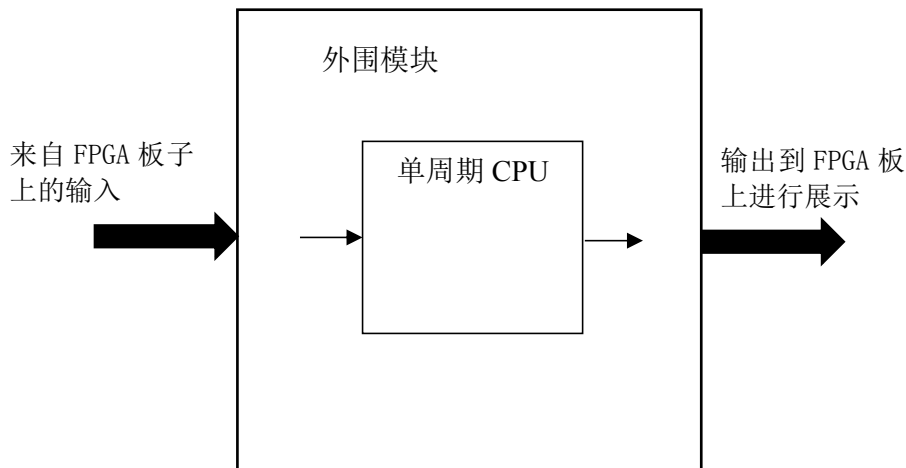


图 7.2 单周期 CPU 设计实验的顶层模块大致框图

9. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：

①：MIPS 架构中有延迟槽的设定，其本意是加快流水 CPU 的执行速度，故在单周期 CPU 中该设定毫无意义，反而带来了 CPU 实现上的麻烦，故建议在单周期 CPU 中不考虑延迟槽技术。

②：MIPS 架构中分支和跳转指令参与计算的 PC 值均为延迟槽指令对应的 PC (即分支跳转指令的 PC+4)，而由于单周期不考虑延迟槽，故在实验中分支跳转指令参与计算使用本指令的 PC 值，故跳转的偏移量设置需要注意下。比如一条指令

“beq, r0, r0, #2”在不考虑延迟槽的单周期 CPU 中，其跳转的目标地址为 beq 指令后面的第 2 条。而在考虑延迟槽的流水 CPU 中，其跳转的目标地址为 beq 指令后面的第 3 条（即延迟槽指令后面的第 2 条）。这里需要理解清楚。

4 实验要求

1. 做好预习：

- 1) 熟知 MIPS 指令类型，深入理解常用指令的功能和编码；
- 2) 归纳常用的 MIPS 指令，确定自己准备实现的 MIPS 指令；
- 3) 对准备实现的指令进行分析，完成表 7.1 的填写；
- 4) 设计本次实验的方案，画出实验方案的设计框图，即补充完善图 7.1；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 7.2；
- 6) 依据自己设计中实现的指令，编写一段不少于 20 行的汇编程序，要求包含所有实现的指令，完成表 7.2 的填写。

2. 实验实施：

- 1) 确认单周期 CPU 的设计框图的正确性；
- 2) 编写 verilog 代码，将表 7.2 中自己编写的汇编程序翻译为二进制，内嵌到指令 ROM 中；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用单周期 CPU 的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先解读表 7.2 中自己编写的汇编程序，然后采用手动输入时钟，每个周期查看 CPU 状态，按照检查人员的要求进行演示，检查指令运行结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

单周期 CPU 能够执行 MIPS 指令集系统的一个子集，共 16 条指令，包括存储访问指令、运算指令、跳转指令。选取的指令集将会用到各个数据通路部件，并体现控制部分设计的关键原理，其余暂未支持的指令的实现方式类似，并将在后续多周期和流水 CPU 实验中陆续加入。

5.1 实验原理

表 7.3 为单周期 CPU 实现的指令集。表 7.4 为这些指令的解析。

表 7.3 单周期 CPU 实现的指令

指令名称	汇编指令	功能描述
无符号加法	addu rd,rs,rt	$GPR[rd] = GPR[rs] + GPR[rt]$
无符号减法	subu rd,rs,rt	$GPR[rd] = GPR[rs] - GPR[rt]$
有符号比较, 小于置位	slt rd,rs,rt	$GPR[rd] = (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$
按位与	and rd,rs,rt	$GPR[rd] = GPR[rs] \& GPR[rt]$
按位或非	nor rd,rs,rt	$GPR[rd] = \sim(GPR[rs] GPR[rt])$
按位或	or rd,rs,rt	$GPR[rd] = GPR[rs] GPR[rt]$
按位异或	xor rd,rs,rt	$GPR[rd] = GPR[rs] \wedge GPR[rt]$
逻辑左移	sll rd,rt,shf	$GPR[rd] = \text{zero}(GPR[rt]) \ll \text{shf}$
逻辑右移	srl rd,rt,shf	$GPR[rd] = \text{zero}(GPR[rt]) \gg \text{shf}$
立即数无符号加法	addiu rt,rs,imm	$GPR[rt] = GPR[rs] + \text{sign_ext}(\text{imm})$
立即数装载高位	lui rt,imm	$GPR[rt] = \{\text{imm}, 16'd0\}$
从内存装载字	lw rt,offset(base)	$GPR[rt] = \text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})]$
向内存存储字	sw rt,offset(base)	$\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})] = GPR[rt]$
判断相等跳转	beq rs,rt,offset	if $GPR[rs] = GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$
判断不等跳转	bne rs,rt,offset	if $GPR[rs] \neq GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$
直接跳转	j target	$PC = \{PC[31:28], \text{target} \ll 2\}$

表 7.4 单周期 CPU 实现的 mips 指令特性归纳

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	addu rd,rs,rt	000000 rs rt rd 0000 100001	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] + GPR[rt]$
	subu rd,rs,rt	000000 rs rt rd 0000 100011	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] - GPR[rt]$
	slt rd,rs,rt	000000 rs rt rd 0000 101010	[rs]	[rt]		rd	$GPR[rd] = (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$
	and rd,rs,rt	000000 rs rt rd 0000 100100	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \& GPR[rt]$
	nor rd,rs,rt	000000 rs rt rd 0000 100111	[rs]	[rt]		rd	$GPR[rd] = \sim(GPR[rs] GPR[rt])$
	or rd,rs,rt	000000 rs rt rd 0000 100101	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] GPR[rt]$
	xor rd,rs,rt	000000 rs rt rd 0000 100110	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \wedge GPR[rt]$
	sll rd,rt,shf	000000 000000 rt rd shf 000000		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \ll \text{shf}$
	srl rd,rt,shf	000000 000000 rt rd shf 000010		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \gg \text{shf}$
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = GPR[rs] + \text{sign_ext}(\text{imm})$
	beq rs,rt,offset	000100 rs rt offset	[rs]	[rt]			if $GPR[rs] = GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$
	bne rs,rt,offset	000101 rs rt offset	[rs]	[rt]			if $GPR[rs] \neq GPR[rt]$ then $PC = PC + \text{sign_ext}(\text{offset}) \ll 2$

表 7.4 单周期 CPU 实现的 mips 指令特性归纳（续）

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
I 型指令	lw rt,offset(b)	100011 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=Mem[GPR[b]+sign_ext(offset)]
	sw rt,offset(b)	101011 b rt offset	[b]	sign_ext (offset)	[rt]		Mem[GPR[b]+sign_ext(offset)]=GPR[rt]
I 型指令	lui rt,imm	001111 00000 rt i mm		{imm, 16'd0}		rt	GPR[rt]= {imm, 16'd0}
J 型指令	j target	000010 target					PC={PC[31:28],target<<2}

单周期 CPU 的实现框图如下：

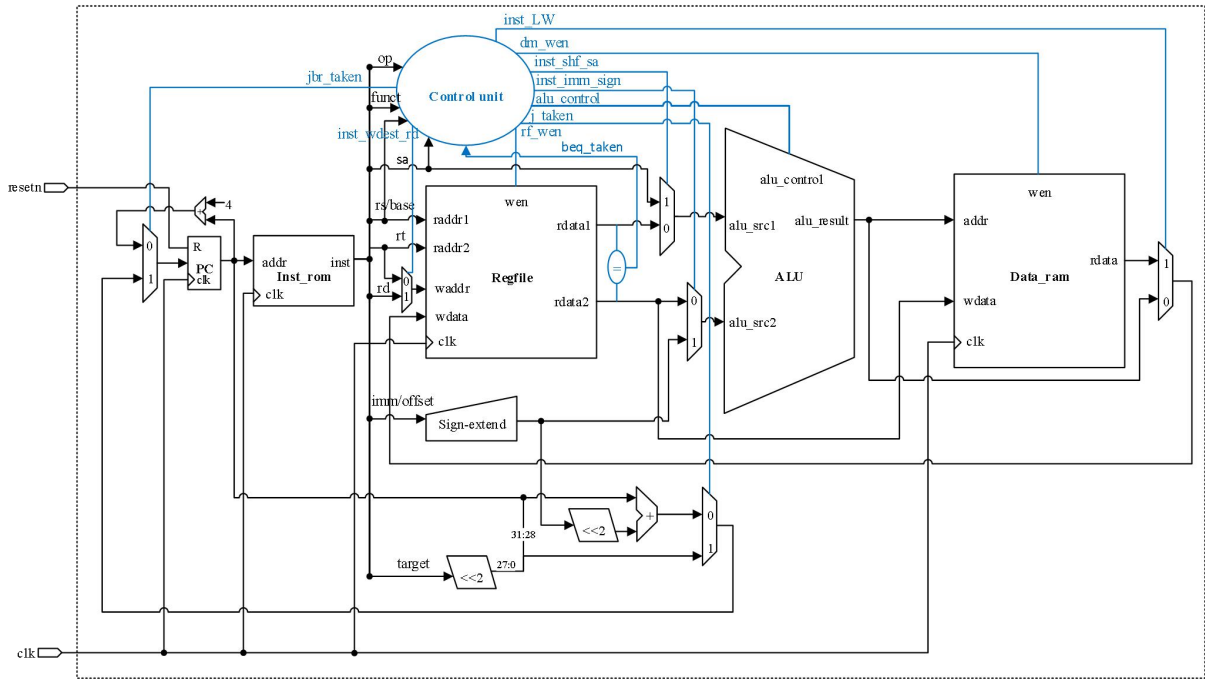


图 7.3 单周期 CPU 的实现框图

利用其实现的 16 条指令，可以编写一段汇编程序，用于测试实现的单周期 CPU 的正确性。表 7.5 给出了一个测试程序。

表 7.5 单周期 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1,\$0,#1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001
04H	sll \$2,\$1,#4	[\$2] = 0000_0010H	00011100	0000_0000_0000_0001_0001_0001_0000_0000
08H	addu \$3,\$2,\$1	[\$3] = 0000_0011H	00411821	0000_0000_0100_0001_0001_1000_0010_0001
0CH	srl \$4,\$2,#2	[\$4] = 0000_0004H	00022082	0000_0000_0000_0010_0010_0000_1000_0010
10H	subu \$5,\$3,\$4	[\$5] = 0000_000DH	00642823	0000_0000_0110_0100_0010_1000_0010_0011
14H	sw \$5,#19(\$1)	Mem[0000_0014H] = 0000_000DH	AC250013	1010_1100_0010_0101_0000_0000_0001_0011
18H	nor \$6,\$5,\$2	[\$6] = FFFF_FFE2H	00A23027	0000_0000_1010_0010_0011_0000_0010_0111
1CH	or \$7,\$6,\$3	[\$7] = FFFF_FFF3H	00C33825	0000_0000_1100_0011_0011_1000_0010_0101

表 7.5 单周期 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
20H	xor \$8,\$7,\$6	[\$8] = 0000_0011H	00E64026	0000_0000_1110_0110_0100_0000_0010_0110
24H	sw \$8,#28(\$0)	Mem[0000_001CH] = 0000_0011H	AC08001C	1010_1100_0000_1000_0000_0000_0001_1100
28H	slt \$9,\$6,\$7	[\$9] = 0000_0001H	00C7482A	0000_0000_1100_0111_0100_1000_0010_1010
2CH	beq \$9,\$1,#2	跳转到指令 34H	11210002	0001_0001_0010_0001_0000_0000_0000_0010
30H	addiu \$1,\$0,#4	不执行	24010004	0010_0100_0000_0001_0000_0000_0000_0100
34H	lw \$10,#19(\$1)	[\$10] = 0000_000DH	8C2A0013	1000_1100_0010_1010_0000_0000_0001_0011
38H	bne \$10,\$5,#3	不跳转	15450003	0001_0101_0100_0101_0000_0000_0000_0011
3CH	and \$11,\$2,\$1	[\$11] = 0000_0000H	00415824	0000_0000_0100_0001_0101_1000_0010_0100
40H	sw \$11,#28(\$0)	Mem[0000_001CH] = 0000_0000H	AC0B001C	1010_1100_0000_1011_0000_0000_0001_1100
44H	sw \$4,#16(\$0)	Mem[0000_0010H] = 0000_0004H	AC040010	1010_1100_0000_0100_0000_0000_0001_0000
48H	lui \$12,#12	[\$12] = 000C_0000H	3C0C000C	0011_1100_0000_1100_0000_0000_0000_1100
4CH	j 00H	跳转指令 00H	08000000	0000_1000_0000_0000_0000_0000_0000_0000

5.2 实验顶层模块框图

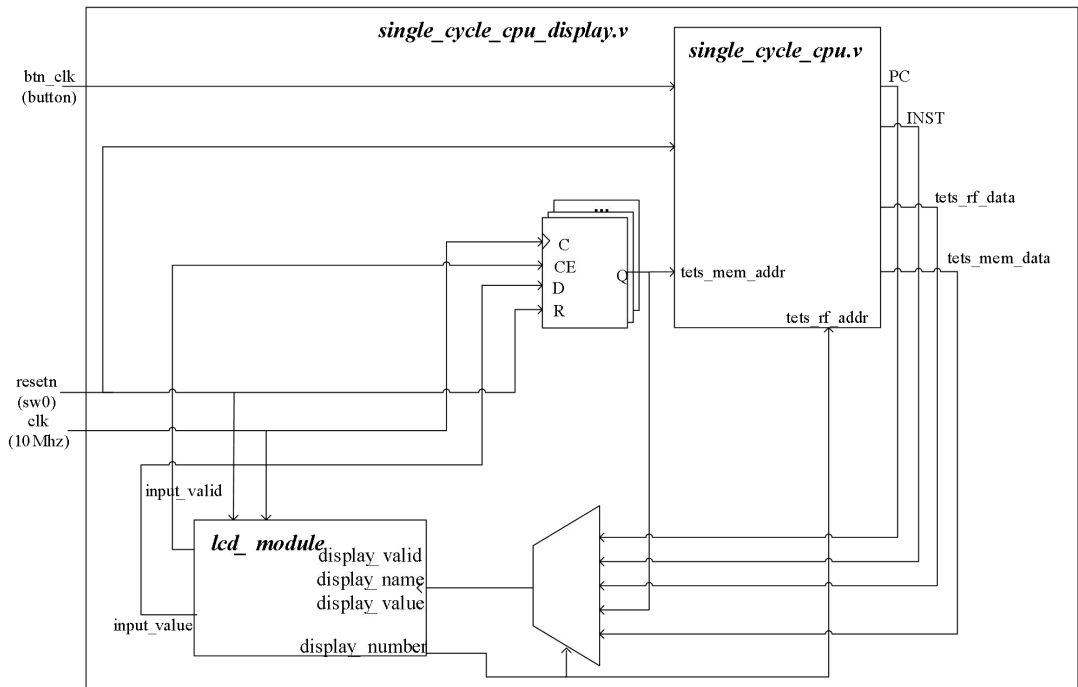


图 7.4 单周期 CPU 参考设计的顶层模块框图

5.3 源代码

single_cycle_cpu_display.v 源代码:

```
1 `timescale 1ns / 1ps
2 //*****
3 // > 文件名: single_cycle_cpu_display.v
4 // > 描述 : 单周期 CPU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5 // > 作者 : LOONGSON
```

```
6 // > 日期 : 2016-04-14
7 //*****
8 module single_cycle_cpu_display(
9     //时钟与复位信号
10     input clk,
11     input resetn, //后缀"n"代表低电平有效
12
13     //脉冲开关, 用于产生脉冲 clk, 实现单步执行
14     input btn_clk,
15
16     //触摸屏相关接口, 不需要更改
17     output lcd_rst,
18     output lcd_cs,
19     output lcd_rs,
20     output lcd_wr,
21     output lcd_rd,
22     inout[15:0] lcd_data_io,
23     output lcd_bl_ctr,
24     inout ct_int,
25     inout ct_sda,
26     output ct_scl,
27     output ct_rstn
28 );
29 //-----{时钟和复位信号}begin
30 //不需要更改, 用于单步调试
31     wire cpu_clk; //单周期 CPU 里使用脉冲开关作为时钟, 以实现单步执行
32     reg btn_clk_r1;
33     reg btn_clk_r2;
34     always @(posedge clk)
35     begin
36         if (!resetn)
37         begin
38             btn_clk_r1<= 1'b0;
39         end
40         else
41         begin
42             btn_clk_r1 <= ~btn_clk;
43         end
44
45         btn_clk_r2 <= btn_clk_r1;
46     end
47
48     wire clk_en;
49     assign clk_en = !resetn || (!btn_clk_r1 && btn_clk_r2);
50     BUFGCE cpu_clk_cg(.I(clk),.CE(clk_en),.O(cpu_clk));
51 //-----{时钟和复位信号}end
52
53 //-----{调用单周期 CPU 模块}begin
54
55     //用于在 FPGA 板上显示结果
```

```

56     wire [31:0] cpu_pc;           //CPU 的 PC
57     wire [31:0] cpu_inst;        //该 PC 取出的指令
58     wire [ 4:0] rf_addr;         //扫描寄存器堆的地址
59     wire [31:0] rf_data;         //寄存器堆从调试端口读出的数据
60     reg  [31:0] mem_addr;        //要观察的内存地址
61     wire [31:0] mem_data;        //内存地址对应的数据
62     single_cycle_cpu cpu(
63         .clk      (cpu_clk      ),
64         .resetn    (resetn      ),
65
66         .rf_addr   (rf_addr   ),
67         .mem_addr  (mem_addr  ),
68         .rf_data   (rf_data   ),
69         .mem_data  (mem_data  ),
70         .cpu_pc    (cpu_pc    ),
71         .cpu_inst  (cpu_inst  )
72     );
73     //-----{调用单周期 CPU 模块}end
74
75     //-----{调用触摸屏模块}begin-----//
76     //-----{实例化触摸屏}begin
77     //此小节不需要更改
78     reg          display_valid;
79     reg  [39:0] display_name;
80     reg  [31:0] display_value;
81     wire [5 :0] display_number;
82     wire          input_valid;
83     wire [31:0] input_value;
84
85     lcd_module lcd_module(
86         .clk      (clk          ), //10Mhz
87         .resetn    (resetn      ),
88
89         //调用触摸屏的接口
90         .display_valid (display_valid ),
91         .display_name  (display_name  ),
92         .display_value (display_value ),
93         .display_number (display_number),
94         .input_valid   (input_valid   ),
95         .input_value   (input_value   ),
96
97         //lcd 触摸屏相关接口, 不需要更改
98         .lcd_rst      (lcd_rst      ),
99         .lcd_cs       (lcd_cs       ),
100        .lcd_rs       (lcd_rs       ),
101        .lcd_wr       (lcd_wr       ),
102        .lcd_rd       (lcd_rd       ),
103        .lcd_data_io   (lcd_data_io   ),
104        .lcd_bl_ctr    (lcd_bl_ctr    ),
105        .ct_int        (ct_int        ),
106        .ct_sda        (ct_sda        ),

```

```
107         .ct_scl          (ct_scl          ),
108         .ct_rstn         (ct_rstn         )
109     );
110 //-----{实例化触摸屏}end
111
112 //-----{从触摸屏获取输入}begin
113 //根据实际需要输入的数修改此小节,
114 //建议对每一个数的输入, 编写单独一个 always 块
115     always @(posedge clk)
116     begin
117         if (!resetn)
118         begin
119             mem_addr <= 32'd0;
120         end
121         else if (input_valid)
122         begin
123             mem_addr <= input_value;
124         end
125     end
126     assign rf_addr = display_number-6'd5;
127 //-----{从触摸屏获取输入}end
128
129 //-----{输出到触摸屏显示}begin
130 //根据需要显示的数修改此小节,
131 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
132 //44 块显示区域从 1 开始编号, 编号为 1~44,
133     always @(posedge clk)
134     begin
135         if (display_number >6'd4 && display_number <6'd37 )
136         begin //块号 5~36 显示 32 个通用寄存器的值
137             display_valid <= 1'b1;
138             display_name[39:16] <= "REG";
139             display_name[15: 8] <= {4'b0011,3'b000,rf_addr[4]};
140             display_name[7 : 0] <= {4'b0011,rf_addr[3:0]};
141             display_value      <= rf_data;
142         end
143         else
144         begin
145             case(display_number)
146             6'd1 : //显示 PC 值
147             begin
148                 display_valid <= 1'b1;
149                 display_name  <= "  PC";
150                 display_value <= cpu_pc;
151             end
152             6'd2 : //显示 PC 取出的指令
153             begin
154                 display_valid <= 1'b1;
155                 display_name  <= " INST";
156                 display_value <= cpu_inst;
157             end
```

```

158             6'd3 : //显示要观察的内存地址
159             begin
160                 display_valid <= 1'b1;
161                 display_name  <= "MADDR";
162                 display_value <= mem_addr;
163             end
164             6'd4 : //显示该内存地址对应的数据
165             begin
166                 display_valid <= 1'b1;
167                 display_name  <= "MDATA";
168                 display_value <= mem_data;
169             end
170             default :
171             begin
172                 display_valid <= 1'b0;
173             end
174         endcase
175     end
176 end
177 //-----{输出到触摸屏显示}end
178 //-----{调用触摸屏模块}end-----//
179 endmodule

```

single_cycle_cpu.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: single_cycle_cpu.v
4  //  > 描述   :单周期 CPU 模块,共实现 16 条指令
5  //  >         指令 rom 和数据 ram 均采用异步读数据,以便单周期 CPU 好实现
6  //  > 作者   : LOONGSON
7  //  > 日期   : 2016-04-14
8  //*****
9  `define STARTADDR 32'd0 // 程序起始地址
10 module single_cycle_cpu(
11     input clk,           // 时钟
12     input resetn,        // 复位信号,低电平有效
13
14     //display data
15     input  [4:0] rf_addr,
16     input  [31:0] mem_addr,
17     output [31:0] rf_data,
18     output [31:0] mem_data,
19     output [31:0] cpu_pc,
20     output [31:0] cpu_inst
21 );
22
23 //-----{取指}begin-----
24 //
25     reg  [31:0] pc;
26     wire [31:0] next_pc;

```

```
26    wire [31:0] seq_pc;
27    wire [31:0] jbr_target;
28    wire jbr_taken;
29
30    // 下一指令地址: seq_pc=pc+4
31    assign seq_pc[31:2] = pc[31:2] + 1'b1;
32    assign seq_pc[1:0] = pc[1:0];
33    // 新指令: 若指令跳转, 为跳转地址; 否则为下一指令
34    assign next_pc = jbr_taken ? jbr_target : seq_pc;
35    always @ (posedge clk) // PC 程序计数器
36    begin
37        if (!resetn) begin
38            pc <= `STARTADDR; // 复位, 取程序起始地址
39        end
40        else begin
41            pc <= next_pc; // 不复位, 取新指令
42        end
43    end
44
45    wire [31:0] inst_addr;
46    wire [31:0] inst;
47    assign inst_addr = pc; // 指令地址: 指令长度 32 位
48    inst_rom inst_rom_module( // 指令存储器
49        .addr (inst_addr[6:2]), // I, 5, 指令地址
50        .inst (inst) // O, 32, 指令
51    );
52    assign cpu_pc = pc; //display pc
53    assign cpu_inst = inst;
54    //-----{取指}end-----
55    //
56    //-----{译码}begin-----
57    //
58    wire [5:0] op;
59    wire [4:0] rs;
60    wire [4:0] rt;
61    wire [4:0] rd;
62    wire [4:0] sa;
63    wire [5:0] funct;
64    wire [15:0] imm;
65    wire [15:0] offset;
66    wire [25:0] target;
67
68    assign op = inst[31:26]; // 操作码
69    assign rs = inst[25:21]; // 源操作数 1
70    assign rt = inst[20:16]; // 源操作数 2
71    assign rd = inst[15:11]; // 目标操作数
72    assign sa = inst[10:6]; // 特殊域, 可能存放偏移量
73    assign funct = inst[5:0]; // 功能码
74    assign imm = inst[15:0]; // 立即数
75    assign offset = inst[15:0]; // 地址偏移量
```

```

75     assign target = inst[25:0];    // 目标地址
76
77     wire op_zero;    // 操作码全 0
78     wire sa_zero;    // sa 域全 0
79     assign op_zero = ~(|op);
80     assign sa_zero = ~(|sa);
81
82     // 实现指令列表
83     wire inst_ADDU, inst_SUBU , inst_SLT, inst_AND;
84     wire inst_NOR , inst_OR , inst_XOR, inst_SLL;
85     wire inst_SRL , inst_ADDIU, inst_BEQ, inst_BNE;
86     wire inst_LW , inst_SW , inst_LUI, inst_J;
87
88     assign inst_ADDU = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
89     assign inst_SUBU = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
90     assign inst_SLT = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
91     assign inst_AND = op_zero & sa_zero & (funct == 6'b100100); // 逻辑与
92     assign inst_NOR = op_zero & sa_zero & (funct == 6'b100111); // 逻辑或非
93     assign inst_OR = op_zero & sa_zero & (funct == 6'b100101); // 逻辑或
94     assign inst_XOR = op_zero & sa_zero & (funct == 6'b100110); // 逻辑异或
95     assign inst_SLL = op_zero & (rs==5'd0) & (funct == 6'b000000); // 逻辑左移
96     assign inst_SRL = op_zero & (rs==5'd0) & (funct == 6'b000010); // 逻辑右移
97     assign inst_ADDIU = (op == 6'b001001); // 立即数无符号加法
98     assign inst_BEQ = (op == 6'b000100); // 判断相等跳转
99     assign inst_BNE = (op == 6'b000101); // 判断不等跳转
100    assign inst_LW = (op == 6'b100011); // 从内存装载
101    assign inst_SW = (op == 6'b101011); // 向内存存储
102    assign inst_LUI = (op == 6'b001111); // 立即数装载高半字节
103    assign inst_J = (op == 6'b000010); // 直接跳转
104
105    // 无条件跳转判断
106    wire j_taken;
107    wire [31:0] j_target;
108    assign j_taken = inst_J;
109
110    assign j_target = {pc[31:28], target, 2'b00};
111
112    //分支跳转
113    wire beq_taken;
114    wire bne_taken;
115    wire [31:0] br_target;
116    assign beq_taken = (rs_value == rt_value); // BEQ 跳转条件: GPR[rs]=GPR[rt]
117    assign bne_taken = ~beq_taken; // BNE 跳转条件: GPR[rs]≠GPR[rt]
118    assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
119    assign br_target[1:0] = pc[1:0]; // 分支跳转目标地址: PC=PC+offset<<2
120
121    //跳转指令的跳转信号和跳转目标地址
122    assign jbr_taken = j_taken // 指令跳转: 无条件跳转 或 满足分支跳转条
件
123    | inst_BEQ & beq_taken
124    | inst_BNE & bne_taken;

```

```

125     assign jbr_target = j_taken ? j_target : br_target;
126
127     // 寄存器堆
128     wire rf_wen;
129     wire [4:0] rf_waddr;
130     wire [31:0] rf_wdata;
131     wire [31:0] rs_value, rt_value;
132
133     regfile rf_module(
134         .clk      (clk      ), // I, 1
135         .wen      (rf_wen   ), // I, 1
136         .raddr1   (rs       ), // I, 5
137         .raddr2   (rt       ), // I, 5
138         .waddr    (rf_waddr ), // I, 5
139         .wdata    (rf_wdata ), // I, 32
140         .rdata1   (rs_value ), // O, 32
141         .rdata2   (rt_value ), // O, 32
142
143         //display rf
144         .test_addr(rf_addr),
145         .test_data(rf_data)
146     );
147
148
149     // 传递到执行模块的 ALU 源操作数和操作码
150     wire inst_add, inst_sub, inst_slt, inst_sltu;
151     wire inst_and, inst_nor, inst_or, inst_xor;
152     wire inst_sll, inst_srl, inst_sra, inst_lui;
153     assign inst_add = inst_ADDU | inst_ADDIU | inst_LW | inst_SW; // 做加法运算
154     assign inst_sub = inst_SUBU; // 减法
155     assign inst_slt = inst_SLT; // 小于置位
156     assign inst_sltu = 1'b0; // 暂未实现
157     assign inst_and = inst_AND; // 逻辑与
158     assign inst_nor = inst_NOR; // 逻辑或非
159     assign inst_or = inst_OR; // 逻辑或
160     assign inst_xor = inst_XOR; // 逻辑或非
161     assign inst_sll = inst_SLL; // 逻辑左移
162     assign inst_srl = inst_SRL; // 逻辑右移
163     assign inst_sra = 1'b0; // 暂未实现
164     assign inst_lui = inst_LUI; // 立即数装载高位
165
166     wire [31:0] sext_imm;
167     wire inst_shf_sa; //使用 sa 域作为偏移量的指令
168     wire inst_imm_sign; //对立即数作符号扩展的指令
169     assign sext_imm = {{16{imm[15]}}, imm}; // 立即数符号扩展
170     assign inst_shf_sa = inst_SLL | inst_SRL;
171     assign inst_imm_sign = inst_ADDIU | inst_LUI | inst_LW | inst_SW;
172
173     wire [31:0] alu_operand1;
174     wire [31:0] alu_operand2;
175     wire [11:0] alu_control;

```

```

176     assign alu_operand1 = inst_shf_sa ? {27'd0,sa} : rs_value;
177     assign alu_operand2 = inst_imm_sign ? sext_imm : rt_value;
178     assign alu_control = {inst_add,          // ALU 操作码, 独热编码
179                          inst_sub,
180                          inst_slt,
181                          inst_sltu,
182                          inst_and,
183                          inst_nor,
184                          inst_or,
185                          inst_xor,
186                          inst_sll,
187                          inst_srl,
188                          inst_sra,
189                          inst_lui};
190 //-----{译码}end-----
191 //
192 //-----{执行}begin-----
193 //
194     wire [31:0] alu_result;
195     alu alu_module(
196         .alu_control (alu_control ), // I, 12, ALU 控制信号
197         .alu_src1     (alu_operand1), // I, 32, ALU 操作数 1
198         .alu_src2     (alu_operand2), // I, 32, ALU 操作数 2
199         .alu_result   (alu_result )  // O, 32, ALU 结果
200     );
201 //-----{执行}end-----
202 //
203 //-----{访存}begin-----
204 //
205     wire [3 :0] dm_wen;
206     wire [31:0] dm_addr;
207     wire [31:0] dm_wdata;
208     wire [31:0] dm_rdata;
209     assign dm_wen  = {4{inst_SW}} & resetn; // 内存写使能,非 resetn 状态下有
210     assign dm_addr = alu_result;            // 内存写地址,为 ALU 结果
211     assign dm_wdata = rt_value;            // 内存写数据,为 rt 寄存器值
212     data_ram data_ram_module(
213         .clk      (clk          ), // I, 1, 时钟
214         .wen      (dm_wen       ), // I, 1, 写使能
215         .addr     (dm_addr[6:2]), // I, 32, 读地址
216         .wdata    (dm_wdata     ), // I, 32, 写数据
217         .rdata    (dm_rdata     ), // O, 32, 读数据
218         //display mem
219         .test_addr(mem_addr[6:2]),
220         .test_data(mem_data      )
221     );

```

```

222 //-----{访存}end-----
223 //
224 //-----{写回}begin-----
225 //
225     wire inst_wdest_rt;    // 寄存器堆写入地址为 rt 的指令
226
227     assign inst_wdest_rt = inst_ADDIU | inst_LW | inst_LUI;
228     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_AND | inst_NOR
229                          | inst_OR | inst_XOR | inst_SLL | inst_SRL;
230     // 寄存器堆写使能信号, 非复位状态下有效
231     assign rf_wen    = (inst_wdest_rt | inst_wdest_rd) & resetn;
232     assign rf_waddr = inst_wdest_rd ? rd : rt; // 寄存器堆写地址 rd 或 rt
233     assign rf_wdata = inst_LW ? dm_rdata : alu_result; // 写回结果
234 //-----{写回}end-----
235 //
235 endmodule

```

inst_rom.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  // > 文件名: inst_rom.v
4  // > 描述   : 异步指令存储器模块, 采用寄存器搭建而成, 类似寄存器堆
5  // >         内嵌好指令, 只读, 异步读
6  // > 作者   : LOONGSON
7  // > 日期   : 2016-04-14
8  //*****
9  module inst_rom(
10     input      [4 :0] addr, // 指令地址
11     output reg [31:0] inst   // 指令
12 );
13
14     wire [31:0] inst_rom[19:0]; // 指令存储器, 字节地址 7'b000_0000~7'b111_1111
15     //----- 指令编码 -----|指令地址|--- 汇编指令 --|- 指令结果 -----//
16     assign inst_rom[ 0] = 32'h24010001;//00H: addiu $1 , $0, #1 | $1 = 0000_0001H
17     assign inst_rom[ 1] = 32'h00011100;//04H: sll   $2 , $1, #4 | $2 = 0000_0010H
18     assign inst_rom[ 2] = 32'h00411821;//08H: addu  $3 , $2, $1 | $3 = 0000_0011H
19     assign inst_rom[ 3] = 32'h00022082;//0CH: srl   $4 , $2, #2 | $4 = 0000_0004H
20     assign inst_rom[ 4] = 32'h00642823;//10H: subu  $5 , $3, $4 | $5 = 0000_000DH
21     assign inst_rom[ 5] = 32'hAC250013;//14H: sw    $5 , #19($1) | Mem[14H] = DH
22     assign inst_rom[ 6] = 32'h00A23027;//18H: nor   $6 , $5, $2 | $6 = FFFF_FFE2H
23     assign inst_rom[ 7] = 32'h00C33825;//1CH: or    $7 , $6, $3 | $7 = FFFF_FFF3H
24     assign inst_rom[ 8] = 32'h00E64026;//20H: xor   $8 , $7, $6 | $8 = 0000_0011H
25     assign inst_rom[ 9] = 32'hAC08001C;//24H: sw    $8 , #28($0) | Mem[1CH] = 11H
26     assign inst_rom[10] = 32'h00C7482A;//28H: slt   $9 , $6, $7 | $9 = 0000_0001H
27     assign inst_rom[11] = 32'h11210002;//2CH: beq   $9 , $1, #2 | 跳转到指令 34H
28     assign inst_rom[12] = 32'h24010004;//30H: addiu $1 , $0, #4 | 不执行
29     assign inst_rom[13] = 32'h8C2A0013;//34H: lw    $10, #19($1) | $10 = 0000_000DH
30     assign inst_rom[14] = 32'h15450003;//38H: bne   $10, $5, #3 | 不跳转
31     assign inst_rom[15] = 32'h00415824;//3CH: and   $11, $2, $1 | $11 = 0000_0000H

```

```

32     assign inst_rom[16] = 32'hAC0B001C;//40H: sw      $11,#28($0) | Mem[1CH] = 0H
33     assign inst_rom[17] = 32'hAC040010;//44H: sw      $4 ,#16($0) | Mem[10H] = 4H
34     assign inst_rom[18] = 32'h3C0C000C;//48H: lui     $12,#12      | [R12] = C_0000H
35     assign inst_rom[19] = 32'h08000000;//4CH: j       00H          | 跳转指令 00H
36
37     //读指令,取 4 字节
38     always @(*)
39     begin
40         case (addr)
41             5'd0 : inst <= inst_rom[0 ];
42             5'd1 : inst <= inst_rom[1 ];
43             5'd2 : inst <= inst_rom[2 ];
44             5'd3 : inst <= inst_rom[3 ];
45             5'd4 : inst <= inst_rom[4 ];
46             5'd5 : inst <= inst_rom[5 ];
47             5'd6 : inst <= inst_rom[6 ];
48             5'd7 : inst <= inst_rom[7 ];
49             5'd8 : inst <= inst_rom[8 ];
50             5'd9 : inst <= inst_rom[9 ];
51             5'd10: inst <= inst_rom[10];
52             5'd11: inst <= inst_rom[11];
53             5'd12: inst <= inst_rom[12];
54             5'd13: inst <= inst_rom[13];
55             5'd14: inst <= inst_rom[14];
56             5'd15: inst <= inst_rom[15];
57             5'd16: inst <= inst_rom[16];
58             5'd17: inst <= inst_rom[17];
59             5'd18: inst <= inst_rom[18];
60             5'd19: inst <= inst_rom[19];
61             default: inst <= 32'd0;
62         endcase
63     end
64 endmodule

```

data_ram.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: data_mem.v
4  //  > 描述   : 异步数据存储器模块,采用寄存器搭建而成,类似寄存器堆
5  //  >          同步写,异步读
6  //  > 作者   : LOONGSON
7  //  > 日期   : 2016-04-14
8  //*****
9  module data_ram(
10     input          clk,          // 时钟
11     input  [3:0]   wen,          // 字节写使能
12     input  [4:0]   addr,        // 地址
13     input  [31:0]  wdata,       // 写数据
14     output reg [31:0] rdata,    // 读数据

```

```
15
16    //调试端口, 用于读出数据显示
17    input      [4 :0] test_addr,
18    output reg [31:0] test_data
19 );
20    reg [31:0] DM[31:0]; //数据存储器, 字节地址 7'b000_0000~7'b111_1111
21
22    //写数据
23    always @(posedge clk) // 当写控制信号为 1, 数据写入内存
24    begin
25        if (wen[3])
26        begin
27            DM[addr][31:24] <= wdata[31:24];
28        end
29    end
30    always @(posedge clk)
31    begin
32        if (wen[2])
33        begin
34            DM[addr][23:16] <= wdata[23:16];
35        end
36    end
37    always @(posedge clk)
38    begin
39        if (wen[1])
40        begin
41            DM[addr][15: 8] <= wdata[15: 8];
42        end
43    end
44    always @(posedge clk)
45    begin
46        if (wen[0])
47        begin
48            DM[addr][7 : 0] <= wdata[7 : 0];
49        end
50    end
51
52    //读数据, 取 4 字节
53    always @(*)
54    begin
55        case (addr)
56            5'd0 : rdata <= DM[0 ];
57            5'd1 : rdata <= DM[1 ];
58            5'd2 : rdata <= DM[2 ];
59            5'd3 : rdata <= DM[3 ];
60            5'd4 : rdata <= DM[4 ];
61            5'd5 : rdata <= DM[5 ];
62            5'd6 : rdata <= DM[6 ];
63            5'd7 : rdata <= DM[7 ];
64            5'd8 : rdata <= DM[8 ];
65            5'd9 : rdata <= DM[9 ];
```

```
66          5'd10: rdata <= DM[10];
67          5'd11: rdata <= DM[11];
68          5'd12: rdata <= DM[12];
69          5'd13: rdata <= DM[13];
70          5'd14: rdata <= DM[14];
71          5'd15: rdata <= DM[15];
72          5'd16: rdata <= DM[16];
73          5'd17: rdata <= DM[17];
74          5'd18: rdata <= DM[18];
75          5'd19: rdata <= DM[19];
76          5'd20: rdata <= DM[20];
77          5'd21: rdata <= DM[21];
78          5'd22: rdata <= DM[22];
79          5'd23: rdata <= DM[23];
80          5'd24: rdata <= DM[24];
81          5'd25: rdata <= DM[25];
82          5'd26: rdata <= DM[26];
83          5'd27: rdata <= DM[27];
84          5'd28: rdata <= DM[28];
85          5'd29: rdata <= DM[29];
86          5'd30: rdata <= DM[30];
87          5'd31: rdata <= DM[31];
88      endcase
89  end
90  //调试端口，读出特定内存的数据
91  always @(*)
92  begin
93      case (test_addr)
94          5'd0 : test_data <= DM[0 ];
95          5'd1 : test_data <= DM[1 ];
96          5'd2 : test_data <= DM[2 ];
97          5'd3 : test_data <= DM[3 ];
98          5'd4 : test_data <= DM[4 ];
99          5'd5 : test_data <= DM[5 ];
100         5'd6 : test_data <= DM[6 ];
101         5'd7 : test_data <= DM[7 ];
102         5'd8 : test_data <= DM[8 ];
103         5'd9 : test_data <= DM[9 ];
104         5'd10: test_data <= DM[10];
105         5'd11: test_data <= DM[11];
106         5'd12: test_data <= DM[12];
107         5'd13: test_data <= DM[13];
108         5'd14: test_data <= DM[14];
109         5'd15: test_data <= DM[15];
110         5'd16: test_data <= DM[16];
111         5'd17: test_data <= DM[17];
112         5'd18: test_data <= DM[18];
113         5'd19: test_data <= DM[19];
114         5'd20: test_data <= DM[20];
115         5'd21: test_data <= DM[21];
116         5'd22: test_data <= DM[22];
```

```
117         5'd23: test_data <= DM[23];
118         5'd24: test_data <= DM[24];
119         5'd25: test_data <= DM[25];
120         5'd26: test_data <= DM[26];
121         5'd27: test_data <= DM[27];
122         5'd28: test_data <= DM[28];
123         5'd29: test_data <= DM[29];
124         5'd30: test_data <= DM[30];
125         5'd31: test_data <= DM[31];
126     endcase
127 end
128 endmodule
```

单周期 CPU 的 `inst_rom.v` 和 `data_ram.v` 均为自行搭建的异步存储器，也是在实验五中 5.5 和 5.6 小节中给出的源代码。单周期 CPU 的源代码还缺少 `regfile.v` 和 `alu.v`，这两个源文件都已在实验三和实验四中给出，本实验中直接通过“Add Copy of Source”添加到工程中即可。其中 `alu.v` 还调用了实验一中的 `adder.v`。

约束文件 `single_cycle_cpu.xdc` 源代码：

```
1  #时钟信号连接
2  set_property PACKAGE_PIN AC19 [get_ports clk]
3
4  #脉冲开关，用于输入作为复位信号，低电平有效
5  set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7  #脉冲开关，用于输入作为单步执行的 clk
8  set_property PACKAGE_PIN Y5 [get_ports btn_clk]
9
10 set_property IOSTANDARD LVCMOS33 [get_ports clk]
11 set_property IOSTANDARD LVCMOS33 [get_ports resetn]
12 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
13
14 #触摸屏引脚连接
15 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
16 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
17 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
18 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
19 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
20 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
21 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
22 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
23 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
24 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
25 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
26 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
27 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
28 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
29 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
30 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
```

```
31 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
32 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
33 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
34 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
35 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
36 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
37 set_property PACKAGE_PIN L19 [get_ports ct_int]
38 set_property PACKAGE_PIN J24 [get_ports ct_sda]
39 set_property PACKAGE_PIN H21 [get_ports ct_scl]
40 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
41
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
43 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
47 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
67 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
68
69 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
```